

Compte Rendu

Atelier de Révision Web

4 Exercices Pratiques — HTML · CSS · JavaScript

Auteur : Aymane Galibou
Dépôt : github.com/Aymane-Galibou
Date : Mai 2025

Table des Matières

1.	Exercice 1 — Calculatrice Simple (Addition / Multiplication)	3
2.	Exercice 2 — Calculatrice Avancée avec Classe JS	5
3.	Exercice 3 — Gestion des Étudiants (CRUD)	7
4.	Exercice 4 — Gestion des Produits avec Filtres & Tri	10

Introduction

Ce compte rendu présente les quatre exercices réalisés dans le cadre de l'atelier de révision Web. Chaque exercice met en pratique les concepts fondamentaux du développement front-end : manipulation du DOM, gestion des événements, programmation orientée objet en JavaScript, et mise en forme avec CSS moderne.

Technologie	Rôle
HTML5	Structure des pages
CSS3 / Flexbox	Mise en page et styles
JavaScript (ES6+)	Logique applicative
Classes JS / Arrow fn	POO et syntaxe moderne

Exercice 1 — Calculatrice Simple

Objectif : Créer une calculatrice permettant d'additionner ou de multiplier deux nombres via des fonctions fléchées JavaScript.

Concept	Description
Fonctions fléchées	additionner / multiplier définies avec la syntaxe =>
Manipulation DOM	getElementById pour lire les inputs et injecter le résultat
Validation saisie	Vérification des champs vides avant calcul
Paramètre type	1 = addition, 2 = multiplication (switch implicite avec if/else)

■ index.html

HTML

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <title>Calculatrice Moderne</title>
5   <link rel="stylesheet" href="style.css">
6 </head>
7 <body>
8   <div class="calculator-container">
9     <h2>Calculateur Dynamique</h2>
10    <input type="number" id="inputA" placeholder="Nombre a">
11    <input type="number" id="inputB" placeholder="Nombre b">
12    <button onclick="calculate(1)">Additionner</button>
13    <button onclick="calculate(2)">Multiplier</button>
14    <div id="resultats">En attente de calcul...</div>
15  </div>
16  <script src="script.js"></script>
17 </body></html>
```

Structure HTML : deux inputs numériques, deux boutons déclenchant calculate(1) et calculate(2), une div #resultats pour l'affichage.

■ script.js

JS

```

1 // Fonctions fléchées pour les opérations
2 const additionner = (a, b) => a + b;
3 const multiplier = (a, b) => a * b;
4
5 const calculate = (type) => {
6   const inputA = document.getElementById('inputA');
7   const inputB = document.getElementById('inputB');
8   const resultDiv = document.getElementById('resultats');
9
10  if (inputA && inputB && resultDiv) {
11    // Validation : champs vides
12    if (inputA.value==='' || inputB.value==='') {
13      resultDiv.innerHTML = '■ Veuillez remplir les deux champs';
14      return;
15    }
16    const valA = Number(inputA.value);
17    const valB = Number(inputB.value);
18    let finalValue = 0;
19    if (type === 1) finalValue = additionner(valA, valB);
20    else if (type === 2) finalValue = multiplier(valA, valB);
21    resultDiv.innerHTML = finalValue; // Injection résultat
22  }
23 };

```

JavaScript : les fonctions fléchées sont séparées de la logique principale. La fonction `calculate()` reçoit un entier (`type`) pour sélectionner l'opération, lit les inputs, valide, puis injecte le résultat dans le DOM.

■ style.css

CSS

```

1 body { font-family: 'Segoe UI'; background: #f4f6f9;
2       display: flex; justify-content: center; align-items: center;
3       min-height: 100vh; }
4 .calculator-container { background: #fff; max-width: 400px;
5   padding: 35px; border-radius: 12px;
6   box-shadow: 0 4px 20px rgba(0,0,0,0.08); }
7 .input-field { padding: 12px 16px; border: 1px solid #dcdde1;
8   border-radius: 8px; transition: all 0.2s; }
9 .btn-add { background: #2980b9; } /* Bleu – Addition */
10 .btn-mul { background: #27ae60; } /* Vert – Multiplication */
11 #resultats { background: #f8f9fa; border: 1px dashed #dcdde1;
12   border-radius: 8px; min-height: 50px; }

```

CSS : mise en page centrée avec Flexbox, inputs et boutons stylisés avec des effets focus et hover, zone de résultat avec bordure pointillée.

Exercice 2 — Calculatrice Avancée avec Classe JS

Objectif : Évolution de l'exercice 1 — utilisation d'une **classe JavaScript** regroupant les 4 opérations arithmétiques. L'opérateur est sélectionné via un menu déroulant `<select>` et un **switch/case** route vers la bonne méthode.

Nouveauté	Exercice 1	Exercice 2
Architecture	Fonctions libres	Classe Calculate avec méthodes
Opérations	Addition, Multiplication	+ Soustraction, Division
Sélection opérateur	Paramètre entier (1/2)	Menu <code><select></code> + switch/case
Division par zéro	Non gérée	Retourne 'Division Impossible'

■ index.html

HTML

```
1 <div class="input-group">
2   <input type="number" id="inputA" placeholder="Nombre a">
3
4   <!-- Sélecteur opérateur -->
5   <select id="inputOp">
6     <option value="1">+</option>
7     <option value="2">-</option>
8     <option value="3">*</option>
9     <option value="4">/</option>
10  </select>
11
12   <input type="number" id="inputB" placeholder="Nombre b">
13 </div>
14
15 <button onclick="calculate()">Calculate</button>
16 <div id="resultats">En attente de calcul...</div>
```

Structure HTML : ajout d'un entre les deux inputs pour choisir l'opérateur (+, -, *, /).

■ script.js

JS

```

1 // Classe regroupant les 4 opérations
2 class Calculate {
3     addition      = (a, b) => a + b;
4     multiplication= (a, b) => a * b;
5     soustraction  = (a, b) => a - b;
6     division      = (a, b) => (b !== 0 ? a/b : 'Division Impossible');
7 }
8
9 const calc = new Calculate();
10
11 const calculate = () => {
12     const valA = Number(inputA.value);
13     const valB = Number(inputB.value);
14     const op   = inputOp.value; // '1','2','3' ou '4'
15     let finalValue = 0;
16
17     switch (op) { // Routage vers la méthode
18         case '1': finalValue = calc.addition(valA, valB); break;
19         case '2': finalValue = calc.soustraction(valA, valB); break;
20         case '3': finalValue = calc.multiplication(valA, valB); break;
21         case '4': finalValue = calc.division(valA, valB); break;
22         default: finalValue = calc.addition(valA, valB);
23     }
24     resultDiv.innerHTML = finalValue;
25 };

```

JavaScript : la classe *Calculate* encapsule les 4 opérations. La fonction *calculate()* lit la valeur du select (1-4) et utilise un switch/case pour déléguer au bon membre de l'instance *calc*. La division par zéro est gérée avec un opérateur ternaire.

Exercice 3 — Gestion des Étudiants (CRUD)

Objectif : Application de gestion de données avec les opérations **CRUD** (Create, Read, Update, Delete) sur une liste d'étudiants. Interface composée d'un tableau HTML, d'un formulaire d'ajout et d'un formulaire d'édition à apparition conditionnelle.

Opération	Fonction JS	Description
Create	ajouterEtudiant()	Valide et ajoute un étudiant dans le tableau
Read	initTableau()	Génère le HTML du tableau depuis le tableau JS
Update	chargerEtudiant() + sauvegarderModification()	Affiche le formulaire d'édition pré-rempli
Delete	supprimerEtudiant()	Filtre le tableau JS puis rafraîchit l'affichage

■ script.js — Classe & Données initiales

JS

```
1 class Etudiant {
2   constructor(id, nom, prenom) {
3     this.id    = id;
4     this.nom   = nom;
5     this.prenom = prenom;
6   }
7 }
8
9 // Tableau initial (données en mémoire)
10 let etudiants = [
11   new Etudiant(1, 'galibou', 'aymane'),
12   new Etudiant(2, 'Mernissi', 'Reda'),
13   new Etudiant(3, 'Laanigri', 'Rachid')
14 ];
```

La classe Etudiant est un modèle simple avec constructeur. Les données initiales sont stockées dans un tableau JavaScript en mémoire.

■ script.js — initTableau() & ajouterEtudiant()

JS

```

1  const initTableau = () => {
2      const tableBody = document.getElementById('table-body');
3      let tableContent = '';
4      etudiants.forEach((etd) => {
5          tableContent += `<tr>
6              <td>${etd.id}</td> <td>${etd.nom}</td> <td>${etd.prenom}</td>
7              <td>
8                  <button onclick="chargerEtudiant('${etd.id}')">éditer</button>
9                  <button onclick="supprimerEtudiant('${etd.id}')">Supprimer</button>
10             </td></tr>`;
11      });
12      tableBody.innerHTML = tableContent; // Injection dans le DOM
13  };
14
15  const ajouterEtudiant = () => {
16      const id = inputId.value;
17      // Vérification doublon d'ID
18      const index = etudiants.findIndex(e => String(e.id) === String(id));
19      if (index !== -1) { alert('ID dupliqué !'); return; }
20      // Validation champs vides
21      if (!id.trim() || !nom.trim() || !prenom.trim()) {
22          alert('Remplissez tous les champs'); return;
23      }
24      etudiants.push(new Etudiant(id, nom, prenom));
25      initTableau(); // Rafraîchir le tableau
26  };

```

initTableau() utilise des template literals pour générer dynamiquement les lignes TR du tableau. ajouterEtudiant() vérifie les doublons avec `findIndex()` avant l'insertion.

■ script.js — Update & Delete

JS

```

1  // CHARGER (pre-remplir le formulaire d'edition)
2  const chargerEtudiant = (id) => {
3      const etudiant = etudiants.find(e => String(e.id) === String(id));
4      if (etudiant) {
5          document.getElementById('editId').value = etudiant.id;
6          document.getElementById('editNom').value = etudiant.nom;
7          document.getElementById('editPrenom').value = etudiant.prenom;
8          document.getElementById('editFormContainer')
9              .classList.remove('hidden'); // Afficher le formulaire
10     }
11 };
12
13 // SAUVEGARDER la modification
14 const sauvegarderModification = () => {
15     const index = etudiants.findIndex(e => String(e.id) === String(id));
16     if (index !== -1) {
17         etudiants[index].nom = nouveauNom;
18         etudiants[index].prenom = nouveauPrenom;
19         initTableau(); annulerEdition(); // Rafraîchir et masquer
20     }
21 };
22
23 // SUPPRIMER avec filter()
24 const supprimerEtudiant = (id) => {
25     etudiants = etudiants.filter(e => String(e.id) !== String(id));
26     initTableau(); // Mettre à jour le tableau
27 };

```

Update : `chargerEtudiant()` pré-remplit le formulaire et retire la classe CSS 'hidden' pour l'afficher. Delete : `supprimerEtudiant()` utilise `filter()` pour créer un nouveau tableau sans l'élément supprimé.

```
1 .custom-table th {
2   background-color: #2980b9; color: #fff;
3   text-transform: uppercase; letter-spacing: 0.5px; }
4
5 .custom-table tbody tr:nth-child(even) { /* Zebra striping */
6   background-color: #f8fafc; }
7
8 .custom-table tbody tr:hover {
9   background-color: #f1f5f9; transition: 0.2s; }
10
11 .btnOp.update { background: rgba(241,196,15,0.2);
12                 border: 1px solid #f1c40f; color: #d68910; }
13 .btnOp.delete { background: rgba(241,15,15,0.2);
14                 border: 1px solid #f10f0f; color: #d61010; }
15
16 .hidden { display: none !important; } /* Cache le form edit */
```

CSS : zebra striping avec `:nth-child(even)`, boutons éditer/supprimer différenciés par couleur, classe `.hidden` pour affichage conditionnel du formulaire d'édition.

Exercice 4 — Gestion des Produits avec Filtres & Tri

Objectif : Extension de l'exercice 3 appliquée aux produits, avec des fonctionnalités avancées : **filtrage dynamique** par prix et catégorie, **tri par prix** (croissant/décroissant), et **notifications toast** pour le retour utilisateur.

Fonctionnalité	Implémentation	Déclencheur
CRUD Produits	Classe Produit + 4 fonctions	Boutons Ajouter / Éditer / Supprimer
Filtre prix maximum	input oninput → filtrerProduits()	Saisie en temps réel
Filtre catégorie	select onchange → filtrerProduits()	Changement sélection
Tri prix ↑↓	trierParPrix() + variable triCroissant	Clic sur en-tête Prix
Listes déroulantes	lancerListeDeroulante() dynamique	Initialisation + édition
Notification toast	afficherMessageSucces() + CSS transition	Ajout / Édition / Suppression

■ script.js — Classe & filtrerProduits()

JS

```
1 class Produit {
2   constructor(id, nom, prix, categorie) {
3     this.id = id; this.nom = nom;
4     this.prix = prix; this.categorie = categorie;
5   }
6 }
7
8 const categories = ['Électronique', 'Maison', 'Jeux'];
9 let triCroissant = true; // Etat du tri (toggle)
10
11 // Filtrage en temps réel (prix + catégorie)
12 const filtrerProduits = () => {
13   const prixMax = document.getElementById('searchPrix').value;
14   const cat = document.getElementById('searchCategorie').value;
15
16   const produitsFiltres = produits.filter((prod) => {
17     const matchPrix = prixMax===' ' || parseFloat(prod.prix)<=parseFloat(prixMax);
18     const matchCat = cat===' ' || prod.categorie === cat;
19     return matchPrix && matchCat; // AND logique
20   });
21   initTableau(produitsFiltres); // Afficher liste filtrée
22 };
```

filtrerProduits() applique deux conditions en parallèle avec filter(). L'événement oninput garantit un filtrage en temps réel sans bouton de validation.

■ script.js — trierParPrix() & Toast notification

JS

```

1 // TRI par prix avec toggle croissant/décroissant
2 const trierParPrix = () => {
3   triCroissant = !triCroissant; // Inverser l'état
4
5   // Appliquer les filtres actuels avant de trier
6   let listeATrier = produits.filter((prod) => {
7     /* ... mêmes conditions que filtrerProduits() ... */
8   });
9
10  // Tri avec comparaison selon l'état courant
11  listeATrier.sort((a, b) =>
12    triCroissant ? a.prix - b.prix : b.prix - a.prix
13  );
14  initTableau(listeATrier);
15  reglerIcôneTri(); // Met à jour l'icône ■ / ■
16 };
17
18 // TOAST – notification visuelle temporaire
19 const afficherMessageSucces = (message) => {
20   const toast = document.getElementById('toast');
21   toast.textContent = message;
22   toast.classList.add('show'); // Déclenche l'animation CSS
23   setTimeout(() => {
24     toast.classList.remove('show'); // Disparaît après 3s
25   }, 3000);
26 };

```

trierParPrix() préserve les filtres actifs avant de trier, évitant d'afficher des produits hors filtre. Le toast utilise une transition CSS (*opacity* + *translateY*) activée/désactivée via *classList*.

■ style.css — Toast & Search

CSS

```

1 .toast-notification {
2   position: fixed; bottom: 20px; right: 20px;
3   background: #2ecc71; color: white;
4   opacity: 0; pointer-events: none;
5   transform: translateY(20px); /* Départ en bas */
6   transition: opacity 0.4s, transform 0.4s;
7 }
8 .toast-notification.show {
9   opacity: 1; transform: translateY(0); /* Remonte */
10 }
11
12 .search-container {
13   background: #f8f9fa; border: 1px solid #e9ecef;
14   border-radius: 8px; margin-bottom: 20px;
15 }
16 .search-fields { display: flex; gap: 20px; }

```

Le toast utilise *transform* + *opacity* pour une animation fluide. La barre de recherche est stylisée séparément du tableau pour une hiérarchie visuelle claire.

■ script.js — lancerListeDeroulante()

JS

```

1 // Popule dynamiquement un <select> avec la liste categories
2 const lancerListeDeroulante = (inputCategorie) => {
3   const select = document.getElementById(inputCategorie);
4   if (select) {
5     select.innerHTML = ''; // Vider avant repopulation
6     const defaultOption = document.createElement('option');
7     defaultOption.value = '';
8     defaultOption.textContent = '-- Sélectionnez --';
9     select.appendChild(defaultOption);
10    categories.forEach((e) => {
11      const option = document.createElement('option');
12      option.value = e; option.textContent = e;
13      select.appendChild(option);
14    });
15  }
16 };
17
18 // Appels d'initialisation au chargement de la page
19 initTableau();
20 lancerListeDeroulante('searchCategorie');
21 lancerListeDeroulante('inputCategorie');

```

lancerListeDeroulante() est réutilisable : elle accepte l'ID du select à peupler. Appelée au chargement pour le filtre ET le formulaire d'ajout, et à nouveau lors de l'édition pour le select du formulaire de modification.

Conclusion & Bilan des compétences

Les quatre exercices forment une progression cohérente allant de la manipulation DOM simple jusqu'à une application CRUD complète avec filtres et animations CSS.

Exercice	Compétences développées	Niveau
Exo 1 — Calculatrice simple	Fonctions fléchées, querySelector, innerHTML	■ ■
Exo 2 — Calculatrice + Classe	Classes JS, méthodes, switch/case, select HTML	■ ■ ■
Exo 3 — Gestion Étudiants CRUD	Classes, find/filter/findIndex, affichage conditionnel	■ ■ ■ ■
Exo 4 — Produits Filtres & Tri	filter() combiné, sort(), CSS transitions, toast UI	■ ■ ■ ■ ■

Dépôt GitHub : <https://github.com/Ayman-Galibou/Atelier-Revision-Web>